



Hands-On Activity: From spreadsheets to BigQuery



Activity Overview

By now, you have worked with data using both spreadsheets and SQL. These tools operate very differently: In spreadsheets, you are able to observe and interact with data directly; with SQL, you interact with data through queries to the database. In this activity, you will use spreadsheets to clean your data before importing it into SQL for analysis.

In this scenario, you have been working for a national store chain as a data analyst. Management is interested in the amount of inventory being kept in storage at regional sites. Your supervisor has asked you to perform an analysis on inventory and sales data to make recommendations for changes to inventory management practices. You have been provided with three datasets containing information about inventory, products, and sales.

By the time you complete this activity, you will be able to combine tools to successfully analyze data. Switching between spreadsheets and SQL can be challenging because they're so different, but once you're more used to both tools, you'll be able to use both more easily. This is important for tackling larger and more complex projects in your career as a data analyst.



To get started, first download the three store data CSV files: inventory, products, and sales.

Before you upload these files to SQL, you can import them into a spreadsheet in Sheets to get comfortable with the data before you start analyzing it in BigQuery. This might not always be possible with larger datasets you encounter in the future, but you should explore as much as possible within this exercise! You can also use this step to perform some data-cleaning tasks.

Follow the instructions to complete each step of the activity. Then answer the questions at the end of the activity before going to the next course item.

If you're using Google Sheets, you'll first need to import the data files into your spreadsheet . Open Sheets and navigate to the **File menu**, then **select Import** from the dropdown list.

File Edit View Insert Format I

Share

% .0



Approvals

C

New



Open

Ctrl+O

Import

Make a copy

Email



Download



Make available offline

Version history



Rename



Move to trash

Publish to the web

Document details



Labels

Spreadsheet settings



Print

Ctrl+P

Select the first file and upload it to the spreadsheet. Choose **Replace spreadsheet** to insert it into the current sheet.

Import file

×

File

Sales.csv

Import location

Replace spreadsheet ▾

Separator type

Detect automatically ▾

☒ Convert text to numbers, dates, and formulas

Import data

Cancel

Then **return to the Import menu** under the **File menu** and **upload the next file**. This time, however, **select Insert new sheet(s)** to create new worksheet tabs with this file.

Import file

×

File

Products.csv

Import location

Separator type

Insert new sheet(s) ▾

Detect automatically ▾

☒ Convert text to numbers, dates, and formulas

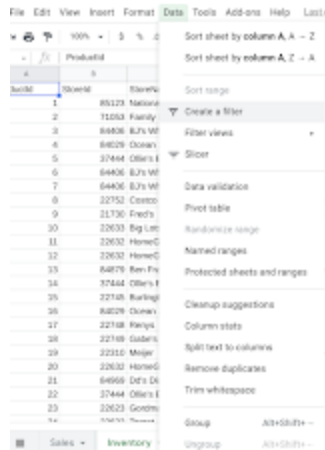
Import data

Cancel

Repeat these steps until you have all three files added to your spreadsheet.

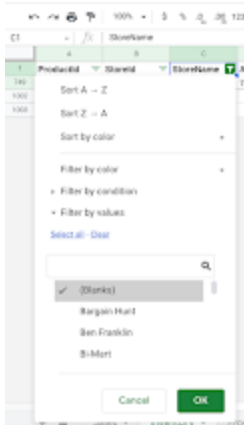
Applying filters in spreadsheets is a good way to identify any data that needs to be cleaned. You'll inspect the Inventory sheet now.

Navigate to the Inventory sheet and **click any cell** in the spreadsheet. **Open the Data dropdown menu** and **select Create a filter**.



Now you can click the filter icons for each column to inspect the values. **Start with the StoreID column.** As you scroll through, you'll notice that there do not appear to be any blanks or incorrectly entered values. However, if you inspect the StoreName column, you'll find a blank.

Deselect all of the values except for the blank.



This should return one row with a missing entry under the StoreName column.

	A	B	C	D	E	F	G	H
1	ProductId	StoreId	StoreName	Address	neighborhood	QuantityAvailable		
749	748	21791		7 Fairfield Drive	Mondawmin	1		
1002								
1003								

You might be able to find what the missing value is and input it correctly using the filter. **Clear the Storename filter** and **use the StoreId column filter** for other stores with the **ID 21791**.

B1		fx	StoreId			
	A	B	C	D	E	F
1	ProductId	StoreId	StoreName	Address	neighborhood	QuantityAvailable
129	128	21791	Dollar Tree	805 Eggendart F	Mondawmin	3
132	131	21791	Dollar Tree	83 South Place	Mondawmin	7
194	193	21791	Dollar Tree	0 Merry Hill	Mondawmin	9
217	216	21791	Dollar Tree	80659 Crownhar	Mondawmin	11
302	301	21791	Dollar Tree	88 Almo Junction	Mondawmin	3
352	351	21791	Dollar Tree	1 Fordem Way	Mondawmin	10
376	375	21791	Dollar Tree	5193 Moland Hil	Mondawmin	2
391	390	21791	Dollar Tree	586 Ruskin Park	Mondawmin	6
440	439	21791	Dollar Tree	52658 Doe Cros	Mondawmin	5
466	465	21791	Dollar Tree	6 Portage Lane	Mondawmin	10
471	470	21791	Dollar Tree	4 Kedzie Parkwa	Mondawmin	4
494	493	21791	Dollar Tree	7311 Southridge	Mondawmin	12
533	532	21791	Dollar Tree	70523 Dixon Pa	Mondawmin	6
593	592	21791	Dollar Tree	6 Commercial Tr	Mondawmin	12
617	616	21791	Dollar Tree	146 Dunning Av	Mondawmin	2
624	623	21791	Dollar Tree	927 Namekagon	Mondawmin	8
686	685	21791	Dollar Tree	1 American Ash	Mondawmin	9
736	735	21791	Dollar Tree	12 Waubesa Pa	Mondawmin	5
747	746	21791	Dollar Tree	3867 Arapahoe I	Mondawmin	4
749	748	21791		7 Fairfield Drive	Mondawmin	1
772	771	21791	Dollar Tree	05 Schurz Circle	Mondawmin	6
793	792	21791	Dollar Tree	2 Katie Point	Mondawmin	2
818	817	21791	Dollar Tree	3987 Hallows Pl	Mondawmin	4
850	849	21791	Dollar Tree	0282 Stephen T	Mondawmin	2

It appears that the other stores with this ID are all Dollar Tree, so it's probably safe to input that as the **StoreName** value in the blank cell.

Inspect the other columns in this sheet, then **return to the Data menu** to turn off the filters. Next, **navigate to the Products sheet**.

Similarly to the last sheet, you can **repeat this process to inspect the Products data**. Go to the Data menu and select Create filter.

Check the ProductID column. You'll find that there is a NA value in this column, despite the fact that this column should only have numeric values. In this case, you've checked in with the dataset owner, who said you can **delete this row** because it was input by mistake and does not belong in this dataset. Turn off the filter and move on to the next step.

From spreadsheets to BigQuery

Applying filters in spreadsheets is an effective way to identify any data that needs to be cleaned. Start by inspecting the Inventory sheet.

1. Navigate to the **Inventory** sheet.

2. Select any non-blank cell.

3. Open the **Data** dropdown menu and select **Create a filter**.

The screenshot shows the Google Sheets interface with the 'Data' menu open. The menu options are: Sort sheet, Sort range, Create a filter (highlighted), Filter views, Add a slicer, Protect sheets and ranges, Named ranges, Named functions (marked 'New'), Randomize range, Column stats, Data validation, Data cleanup (marked with a green dot), Split text to columns, Data extraction, and Data connectors (marked 'New'). The background spreadsheet has columns A, B, and C. Column A is labeled 'ProductId' and column B is labeled 'StoreId'. The data in column B includes values like 85123, 71053, 84406, 84029, 37444, etc. The bottom of the screen shows a tab bar with 'Sales', 'Inventory' (selected), and 'Products'.

ProductId	StoreId	StoreName
1	85123	National
2	71053	Family I
3	84406	BJ's Wh
4	84029	Ocean S
5	37444	Ollie's E
6	84406	BJ's Wh
7	84406	BJ's Wh
8	22752	Costco
9	21730	Fred's
10	22633	Big Lots
11	22632	HomeG
12	22632	HomeG
13	84879	Ben Fra
14	37444	Ollie's E
15	22745	Burlingt
16	84029	Ocean S
17	22748	Renys
18	22749	Gabe's
19	22310	Meijer
20	22632	HomeG

4. Now you can select the filter icons for each column to inspect the values. **Start with the StoreID column.** As you scroll through, you might notice that there do not appear to be any blanks or incorrectly entered values. However, if you inspect the StoreName column, you'll find a blank.

5. Deselect all of the values except for the blank.

6. Select the **StoreName** filter dropdown

7. Select **Clear**.

8. Select the checkbox next to the **(Blanks)** value in the list.

9. Select **OK**.

	A	B	C	
1	ProductId	StoreId	StoreName	Address
2				9 Spring
3				5434 Da
4				3 Darwi
5				684 Bur
6				50162 J
7				8 Forde
8				89058 M
9				1 Clarer
10				629 Cre
11				175 Kim
12				689 Ver
13				78107 L
14				28413 k
15				828 low
16				28352 T
17				0 Cottor
18				14 Mayl
19				010 Lun
20				0 Dryde
21				92 Heffe
22				634 Jac
23				5 Warne
24				353 Rla

This should return one row with a missing entry under the StoreName column.

	A	B	C	D	E	F
1	ProductId	StoreId	StoreName	Address	neighborhood	QuantityAvailable
749	748	21791		7 Fairfield Drive	Mondawmin	1
1002						
1003						

[Add](#)

 more rows at the bottom

You might be able to find what the missing value is and input it correctly using the filter!

10. Clear the **Storename** filter and use the **StoreId** column filter for other stores with the **ID 21791**.

11. Copy the **StoreID** (21791)

12. Clear the StoreName Filter using the StoreName filter dropdown. Select **Select All** then **OK**.

13. Select the **StoreId** filter dropdown.

14. Select **Clear**.

15. Enter **21791** to the search bar which will pull up that StoreID.

16. Select the checkbox next to the **21791** value in the list

17. Select **OK**.

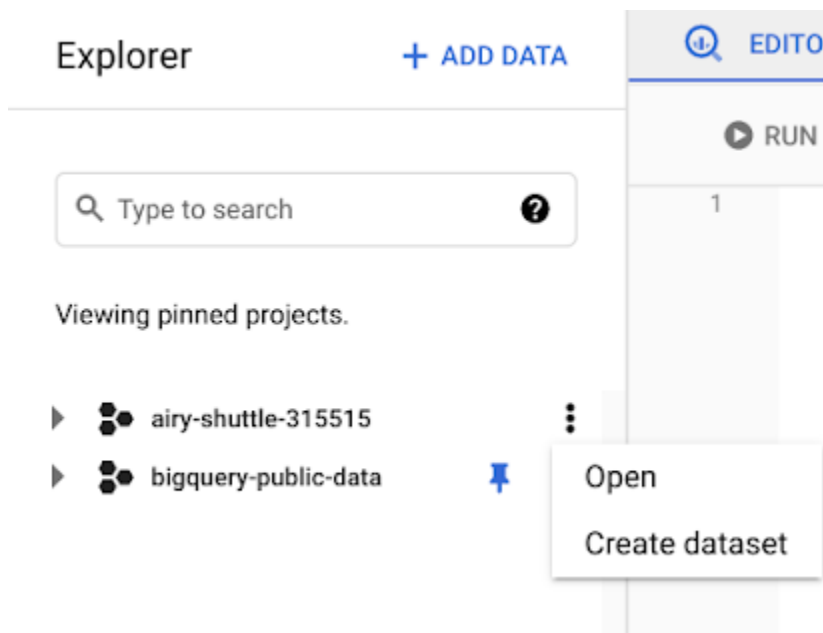
	A	B	C	D	E	F
1	ProductId	StoreId	StoreName	Address	neighborhood	QuantityAvailable
129	128	21791	Dollar Tree	805 Eggendart P	Mondawmin	3
132	131	21791	Dollar Tree	83 South Place	Mondawmin	7
194	193	21791	Dollar Tree	0 Merry Hill	Mondawmin	9
217	216	21791	Dollar Tree	80659 Crownharc	Mondawmin	11
302	301	21791	Dollar Tree	88 Almo Junction	Mondawmin	3
352	351	21791	Dollar Tree	1 Fordem Way	Mondawmin	10
376	375	21791	Dollar Tree	5193 Moland Hill	Mondawmin	2
391	390	21791	Dollar Tree	586 Ruskin Park	Mondawmin	6
440	439	21791	Dollar Tree	52658 Doe Cross	Mondawmin	5
466	465	21791	Dollar Tree	6 Portage Lane	Mondawmin	10
471	470	21791	Dollar Tree	4 Kedzie Parkwa	Mondawmin	4
494	493	21791	Dollar Tree	7311 Southridge	Mondawmin	12
533	532	21791	Dollar Tree	70523 Dixon Par	Mondawmin	6
593	592	21791	Dollar Tree	6 Commercial Tr	Mondawmin	12
617	616	21791	Dollar Tree	146 Dunning Ave	Mondawmin	2
624	623	21791	Dollar Tree	927 Namekagon	Mondawmin	8
686	685	21791	Dollar Tree	1 American Ash	Mondawmin	9
736	735	21791	Dollar Tree	12 Waubesa Par	Mondawmin	5
747	746	21791	Dollar Tree	3867 Arapahoe F	Mondawmin	4
749	748	21791		7 Fairfield Drive	Mondawmin	1
772	771	21791	Dollar Tree	05 Schurz Circle	Mondawmin	6
793	792	21791	Dollar Tree	2 Katie Point	Mondawmin	2
818	817	21791	Dollar Tree	3987 Hallows Pl	Mondawmin	4

+ ≡ Sales ▾ **Inventory** ▾ Products ▾

Now that you have checked out your data in a tool that lets you observe and interact with your data directly, it's time to transition to using SQL. With SQL, you can only observe the results of your query, which requires a different mindset than spreadsheets — but SQL is very powerful when you're working with databases and larger datasets!

Similar to previous activities, you will need to create a dataset and custom table to house this data before you can inspect it in BigQuery.

1. From the **Explorer pane** in your BigQuery console, click the three vertical dots next to your project space and select **Create dataset**.



2. Name the new dataset sales and leave the other settings as their default. Then click **CREATE DATASET**. The new dataset should appear in your **Explorer pane**.

Create dataset

Dataset ID *

sales

Letters, numbers, and underscores allowed

Data location

United States (us)



Default table expiration

☐ Enable table expiration

Default maximum table age

Days

Encryption

- ☒ Google-managed encryption key
No configuration required
- ☐ Customer-managed encryption key (CMEK)
Manage via Google Cloud Key Management Service

CREATE DATASET

CANCEL

3. Open the new dataset. **Click CREATE TABLE.** This will open a Create table menu. **Select create table from upload** and import your sales data. Name the table **sales_info**, select **Auto detect** under **Schema**, and leave the rest of the options as default. Then select **Create table.**

Create table

Source

Create table from:

Upload

Select file: ?

Sales.csv

Browse

File format:

CSV

Destination

☒ Search for a project

☐ Enter a project name

Project name

test

Dataset name

sales

Table type ?

Native table

Table name

sales_info

Schema

Auto detect

☒ Schema and input parameters

Schema will be automatically generated.

Partition and cluster settings

Partitioning: ?

No partitioning

Clustering order (optional): ?

Clustering order determines the sort order of the data. Clustering can be used on both partitioned and non-partitioned tables.

Comma-separated list of fields to define clustering order (up to 4)

Create table

Cancel

4. Open the new table to inspect the schema and preview your data.

Next, you will need to inspect the data to determine how much of it will be useful for your final analysis.

1. Ensure that the import was successful by running this query:

```
SELECT * FROM `sinuous-anvil-461304-h0.Sales.sales_info` LIMIT 10
```

Your results should appear like this:

Query results

[SAVE RESULTS](#)[EXPLORE DATA](#) ▼

Query complete (1.6 sec elapsed, 9.2 MB processed)

Job information [Results](#) [JSON](#) [Execution details](#)

Row	SalesId	StoreId	ProductId	Date	UnitPrice	Quantity
1	11534	21777	256	2017-02-20	1.4175	5
2	65533	21777	256	2019-08-27	1.4175	31
3	86670	21777	256	2020-03-03	1.4175	100
4	81945	21777	256	2019-09-30	1.4175	79
5	73445	21777	256	2018-05-10	1.4175	24
6	17634	21777	256	2018-03-14	1.4175	40
7	87573	21777	512	2018-10-14	2.24	88
8	63291	21777	512	2018-04-20	2.24	92
9	68049	21777	512	2019-07-21	2.24	45

2. Next, inspect the data to find out how many years of sales data it includes. You can use the MIN and MAX functions to get the oldest and newest dates:

```
SELECT
```

```
    MIN(Date) AS min_date,    MAX(Date)
```

```
FROM `sinuous-anvil-461304-h0.Sales.sales_info` LIMIT 1000
```

Now you know what years this data covers. In this case, you'll want to group the data by month because management wants to see year-over-year changes to inventory by month.

3. Click **COMPOSE NEW QUERY** and run the following query, which will return the total quantity sold for each ProductId grouped by the month and year it was sold:

```
SELECT

    EXTRACT(YEAR FROM date) AS Year,

    EXTRACT(MONTH FROM date) AS Month, ProductId,

    ROUND(MAX(UnitPrice),2) AS UnitPrice,

    SUM(Quantity) AS UnitsSold

FROM `sinuous-anvil-461304-h0.Sales.sales_info`

GROUP BY Year, Month, ProductId

ORDER BY Year, Month, ProductId;
```

The subset of data you queried is fewer than 50,000 rows. This means it can be easily exported to a spreadsheet, if your stakeholder requests the data in this form. Or, you can use this exported spreadsheet for visualization. First, however, you'll need to save your results.

1. After running the query, click **SAVE RESULTS**. There will be a pop-up menu with the option to choose the file type for export. Select **CSV Google Drive**. Once it is downloaded, open the new CSV file in Drive.

Query results

 **SAVE RESULTS**

 **EXPLORE DATA** ▾

2. Open the CSV file with Google Sheets.

bq-results-20210616-150938-19kabmq6fgmn.csv

	A	B	C	D	E
1	Year	Month	ProductId	UnitPrice	TotalSold
2	2017	1	2	5.23	231
3	2017	1	3	0.3	427
4	2017	1	4	9.24	159
5	2017	1	5	1.36	290
6	2017	1	6	0.65	362
7	2017	1	8	2.61	21
8	2017	1	9	4.08	488
9	2017	1	10	0.18	272
10	2017	1	11	1.54	232
11	2017	1	12	5.14	163
12	2017	1	13	2.13	35
13	2017	1	14	2.66	38
14	2017	1	15	10.08	155
15	2017	1	16	1.84	109
16	2017	1	17	0.6	324
17	2017	1	18	1.8	306
18	2017	1	19	0.25	285
19	2017	1	20	6.77	117
20	2017	1	21	4.29	83
21	2017	1	22	2.73	52
22	2017	1	23	0.75	252
23	2017	1	24	3.13	480
24	2017	1	25	8.45	257
25	2017	1	26	0.63	12
26	2017	1	27	1.03	270
27	2017	1	28	0.61	28
28	2017	1	29	4.39	161
29	2017	1	30	3.85	394
30	2017	1	31	1.68	328

Open with ▾

Connected apps

 Anyfile Notepad

 AppSheet

 Google Sheets

 Connect more apps

−

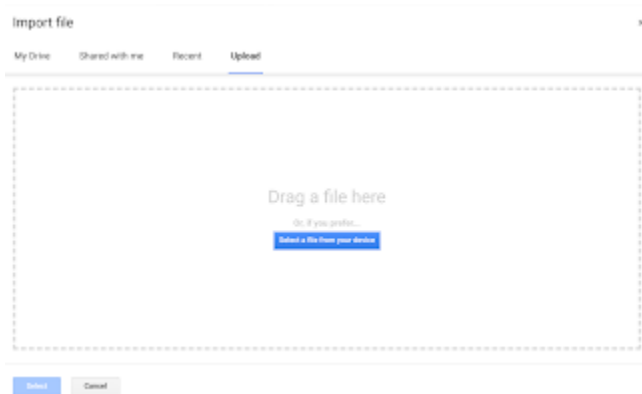
🔍

+

There should be about 47,000 rows. Right-click on the sheet tab and rename the sheet **Sales**.

3. Next, if you're using Sheets, you can open these results by selecting the **File menu** and clicking **Import**.

This will open a pop-up menu. Click **Upload** and select the inventory CSV file.



Select **Insert new sheet(s)** to add this data as a worksheet to your spreadsheet and choose **Comma** for **Separator type**.

Import file



File

Inventory.csv

Import location

Separator type

Insert new sheet(s) ▾

Comma ▾

☒ Convert text to numbers, dates, and formulas

Import data

Cancel

4. Repeat these steps for the productsCSV file.

1 / 1 point

1. What is the earliest year included in this dataset?

☒ 2017

✓ **Correct**

2017 is the earliest year included in this dataset. To find the date range of this dataset, you used MIN and MAX functions in SQL to determine the earliest and latest years. You were able to pull this observation without actually scrolling through all of the data manually, which is a key skill when working with larger datasets.

☐ 2018

☐ 2019

☐ 2020

2. In the text box below, write 2-3 sentences (40-60 words) in response to each of the following questions:

1 / 1 point

- Why is being able to make use of multiple analysis tools useful for some projects?
- How is working with data in spreadsheets and with SQL different? How are they similar?

My Answer: Multiple analysis tools are very useful for some projects because data validation is the important tool for data analysis. If in the raw data there was an error or any missing value, then we can check it from multiple tools. If one tool misses the error, then the other tool might fetch the error, so multiple analysis tools are very useful.

In the spreadsheet we can use a chart to visualize the data, but in SQL it's not possible.

In spreadsheets and SQL we can use similar types of formulas or actions to analyze the data; this is the similarity of these tools.

Feedback

Congratulations on completing this hands-on activity! In this activity, you previewed data in BigQuery to find a useful subset to analyze, imported it to spreadsheets, and analyzed your data! A good response would include that using multiple tools allows you to be more flexible.

Being able to use SQL to create a subset of data to work with in spreadsheets like you did today gives you more options for how you approach your analysis. In upcoming activities, you will have more opportunities to analyze data from beginning to end!